PUBLISHED Apr 08, 2026
ARTICLE UWTN 2026-005
VERSION 1.0.0
LICENCE CC-BY-4.0
ARCHIVED 2026-08-10T04:19:43Z
SOURCE https://www.underworldcode.org/physical-units-in-computational-geodynamics/

Physical Units in Computational Geodynamics

Louis Moresi[1], and **Ben Knight**[2]

[1]Australian National University, [2]Curtin University

DOI https://doi.org/10.6084/m9.figshare.33193587

Abstract

Non-dimensionalisation (rewriting problems to make them re-scalable from lab to the real world) is a specialised task that all modellers confront at some point. It is important for accurate and efficient numerical solutions. Can we take away the pain that comes with the task ?

Keywords Underworld Code, development

Geodynamics involves quantities that span extraordinary ranges. Viscosity might be 10^{21} Pa·s, density 3300 kg/m^3, thermal diffusivity 10^{-6} m^2/s. A single model combines all of these, and the solver needs them in a form where the numbers are close to unity. That means we need to non-dimensionalise our systems of equations. Every user of a geodynamics code does this in some fashion, but most codes force the user to do the book keeping themselves.

Underworld3 handles this differently. You write a model in physical units. The framework tracks those units through the symbolic pipeline, non-dimensionalises everything before it reaches PETSc, and re-dimensionalises the results when you read them back. The solver always works in non-dimensional space, but you never see a non-dimensionalised quantity unless you ask for one.

```
import underworld3 as uw

# Create quantities with physical units
viscosity = uw.quantity(1e21, "Pa*s")
density   = uw.quantity(3300, "kg/m**3")
gravity   = uw.quantity(9.8, "m/s**2")
depth     = uw.quantity(2900, "km")

# These are Pint objects — arithmetic preserves units
buoyancy = density * gravity * depth
buoyancy.units   # <Unit('kilogram / meter / second ** 2 * kilometer')>
buoyancy.to("GPa")  # converts correctly
```

This post describes how the units system works, what it cost us to build, and why we think it was worth it.

1. The Problem with Manual Scaling

In a typical geodynamics workflow without automatic units, you choose reference values for length, time, mass, and temperature. Then you divide every input parameter by the appropriate combination of reference values to produce dimensionless numbers and pass the result to the solver. The solver returns dimensionless solutions. You then have to multiply by the reference values to get physical values.

This works, but it has failure modes that are quiet and expensive. The most common: you non-dimensionalise a quantity with the wrong combination of reference scales, or you forget to non-dimensionalise one parameter entirely. The solver runs. It converges. The answer looks plausible. And it is wrong by a factor of 10^6.

You can guard against this with careful documentation and naming conventions. But every problem requires active attention to refererence quantities and scaling, and discipline in propagating scalings through a script or workflow. Every scale-change is an opportunity for bugs to be introduced.

2. String Input, Pint Object Storage

The user-facing API is simple. `uw.quantity()` takes a number and a unit string (thank you to Pint for consistent conversion) and returns a `UWQuantity` object backed by the Pint library:

```
eta = uw.quantity(1e21, "Pa*s")
eta.value   # 1e+21 — what the user sees
eta.units   # <Unit('pascal * second')>

# Arithmetic works through Pint
kappa = uw.quantity(1e-6, "m**2/s")
time_scale = depth**2 / kappa
time_scale.to("Myr")  # meaningful geological time
```

The principle is: accept strings for convenience, store Pint objects internally. This means every quantity carries its units as metadata, not as part of a naming convention. When you ask `eta.units`, you get a Pint Unit object that knows how to convert, compare, and combine with other unit-aware objects.

A Pint Quantity is a value plus units. You can convert it. A Pint Unit is just the unit, without a value. The distinction matters because `.to()` is a method on quantities, not on units. If you try `eta.units.to("Pa*s")`, you get an `AttributeError`. This catches a real conceptual error: conversion is something we do to a measurement, not to a label.

3. UWexpression: The Symbolic Bridge

Quantities are concrete numbers with units. But Underworld3's solver pipeline works with SymPy expressions that defer evaluation. The bridge between these two worlds is UWexpression.

```
# A UWexpression wraps a quantity with a symbolic name
alpha = uw.expression(r"\alpha", uw.quantity(3e-5, "1/K"), "thermal expansivity")
DeltaT = uw.expression(r"\Delta T", uw.quantity(1500, "K"), "temperature contrast")

# In SymPy, alpha behaves like any symbol
```

```
buoyancy_contribution = alpha * DeltaT   # a SymPy product

# But it knows its units
alpha.units            # <Unit('1 / kelvin')>
buoyancy_ratio.units   # dimensionless — the K cancels
```

The UWexpression is a `SymPy.Symbol` subclass. It participates in symbolic algebra exactly like any other symbol. You can differentiate through it, simplify around it, substitute it. But inside, it holds a reference to its concrete value, and that value carries units.

This is the transparent container principle. A UWexpression does not have its own units. It derives them from whatever it contains. If it wraps a UWQuantity, the units come from the quantity. If it wraps a composite SymPy expression built from other unit-carrying atoms, the units are computed on demand by walking the expression tree.

```
# Units are discovered, not stored
composite = alpha * density * gravity
composite_units = uw.get_units(composite)
# Computed: (1/K) * (kg/m³) * (m/s²) = kg/(K·m²·s²)
```

There is no cached units attribute on composite expressions. Every time you ask for units, the system walks the tree, finds the atomic quantities, and computes the result through Pint arithmetic. This sounds expensive, but it only happens when someone asks. The solver never asks. It works in dimensionless space, where units have already been stripped at the compilation boundary.

4. Derivatives and Units

Spatial derivatives require special treatment. If temperature has units of kelvin and the coordinate has units of metres, then $\partial T/\partial x$ has units of K/m. The units system handles this explicitly.

In Underworld3, spatial derivatives of mesh variables are represented as special SymPy function objects that store a reference back to the parent variable and know which coordinate index they differentiate with respect to. When `get_units()` encounters one of these derivative symbols, it extracts the variable's units and the coordinate's units, and computes the quotient. A velocity gradient $\partial v/\partial x$ with units of (m/s)/m correctly reduces to 1/s. A temperature gradient with units of K/m can be combined with a thermal conductivity in W/(m·K) to produce a heat flux in W/m².

When you write `T.diff(x)` in UW3, the result is not a generic SymPy Derivative object. It is an `UnderworldAppliedFunctionDeriv` that carries metadata about the parent variable and the differentiation coordinate. The units system reads that metadata directly: variable units divided by coordinate units. For the SymPy algebra, the derivative is still a symbol. For the units system, it is a symbol whose units can be computed from first principles.

This matters because constitutive models are built from derivatives. A viscous stress $\sigma = 2\eta\dot{\varepsilon}$ involves the strain rate, which is a velocity gradient. The units of the stress expression are computed by walking the tree: η contributes Pa·s, the strain rate contributes 1/s, and the product gives Pa. If you accidentally define a viscosity with units of kg instead of Pa·s, the units of the stress will not be pascal, and `get_units()` will tell you.

The Jacobian computation that happens inside the solver uses SymPy's `derive_by_array()` to differentiate the residual with respect to the unknown fields and their gradients. This is general symbolic differentiation and does not need to consult the units system at all. By the time these derivatives are used, the expressions have been unwrapped and non-dimensionalised, so units have already been stripped. The two kinds of differentiation live in different parts of the pipeline: spatial derivatives with units exist in the user-facing symbolic layer; Jacobian derivatives exist in the dimensionless compilation layer.

One case we have not yet tested is differentiating with respect to a design parameter for adjoint problems. If you wanted $\partial J/\partial \eta$ where J is a cost functional and η is (say) a viscosity parameter, SymPy's symbolic differentiation would produce the correct expression. The general unit-discovery machinery (`compute_expression_units()`) should be able to walk the resulting expression tree and determine its units and scaling in the equation system. But we have not tested this part of the code explicitly — *(caveat emptor) !*

5. Rendering and Simplifying Units

Pint arithmetic preserves units faithfully, but the results can be verbose. Multiply a density in kg/m^3 by a gravity in m/s^2 by a length in km, and you get units of `kilogram / meter / second ** 2 * kilometer`. Correct, but not what you want to read.

UW3 provides three tools for cleaning this up, all available at the top level:

```
# to_reduced_units: cancel and simplify to named SI units
force = uw.quantity(100, "kg*m/s**2")
uw.to_reduced_units(force)   # 100.0 newton

# to_compact: choose magnitude-appropriate prefixes
length = uw.quantity(1e9, "m")
uw.to_compact(length)        # 1.0 gigameter

# .to(): explicit conversion to a specific unit
pressure = uw.quantity(1e5, "Pa")
pressure.to("bar")           # 1.0 bar
```

`to_reduced_units()` is the workhorse for dimensional analysis. It collapses compound units into their simplest named form. `to_compact()` adjusts prefixes so the number is human-readable. Both work on UWQuantity, UWexpression, UnitAwareArray, and raw Pint objects.

The model also uses magnitude-aware display when reporting reference scales. When you set reference quantities, the summary shows `2900 kilometer` rather than `2.9e6 meter`, and `40 megayear` rather than `1.26e15 second`. These are the same values in different clothes, chosen to match how a geodynamicist thinks about them.

6. The Gateway Pattern

The units system operates at boundaries, not throughout the computation. Three gateways handle all the transitions:

Input gateway — `uw.quantity()` creates dimensional values from user input. This is where units enter the system.

Compilation gateway — when the JIT compiler prepares expressions for C code generation, it unwraps everything. UWexpressions become their non-dimensional `.data` values. UWQuantities become dimensionless floats. Mesh variable symbols become array accessors. By the time SymPy's code printer sees the expression, there are no units left. The C code works entirely with doubles.

Output gateway — `evaluate()` takes the solver's dimensionless results, looks up what units the expression should have, and wraps the result in a `UnitAwareArray` with the correct Pint units attached. The user sees physical numbers.

This means the SymPy algebra layer in the middle never tracks units. It does not need to. The expressions are symbolically correct regardless of what units the atoms carry. The physical correctness is guaranteed by the gateways: dimensional in, dimensionless through the solver, dimensional out.

7. Reference Quantities and Non-Dimensionalisation

To cross the compilation gateway, the system needs reference scales. The user provides these through the model:

```
model = uw.get_default_model()
model.set_reference_quantities(
    domain_depth     = uw.quantity(2900, "km"),
    plate_velocity   = uw.quantity(5, "cm/year"),
    mantle_viscosity = uw.quantity(1e21, "Pa*s"),
    mantle_temperature = uw.quantity(1500, "K"),
)
```

From these, the system derives fundamental scales for length, time, mass, and temperature. Any quantity can then be non-dimensionalised by dividing by the appropriate combination of fundamental scales. Viscosity in Pa·s becomes a number near 1. Length in km becomes a number near 1. The solver works with well-conditioned numbers throughout.

The non-dimensionalisation is automatic and pervasive. When you assign a dimensional value to a mesh variable through `.array`, the unit conversion layer non-dimensionalises it before writing to PETSc. When you read `.array`, it re-dimensionalises. The `.data` property bypasses this and gives you the raw dimensionless values that PETSc stores.

This is why `.array` is the recommended access path for user code. It handles the unit boundary transparently. `.data` is for when you want to work in solver space directly or avoid the conversion overhead.

8. What the Solver Sees

The JIT compiler (described in our SymPy-to-C post) extracts runtime constants from the expression tree. Each UWexpression that has no spatial dependence becomes an entry in a flat constants array. At extraction time, the compiler calls `.data` on each constant, which returns the non-dimensional value.

The C code that PETSc runs looks like this:

```
out[0] = constants[0] * petsc_u_x[0];
// constants[0] is the non-dimensional viscosity — a number as close to 1 as we can make it !
```

If you change a parameter between solves, the constants array is repacked with new non-dimensional values. No recompilation. The structural form of the expression has not changed, only the numbers.

This separation is important for time-stepping problems. The time-step size, continuation parameters, BDF coefficients all change between solves. They are UWexpressions in the symbolic layer, constants in the compiled C, and they update cheaply because the unit conversion and non-dimensionalisation happen at packing time, not at compile time.

9. What This Cost

The units system was the hardest part of the Underworld3 rebuild. We described this in the AI development post — it was Phase 2, the one where context overflowed and sessions drifted.

The difficulty was not in the units library itself. Pint is mature and works well. The difficulty was that units touch everything. The mesh construction code, the variable initialisation, the solver templates, the JIT compiler, the evaluation pathway, the visualisation layer, the checkpoint system. Adding units meant revisiting every module in the codebase.

Two principles eventually brought the work under control.

First: "the user must see every quantity as having units, no exceptions. If a quantity is dimensionless, that is the unit they see." This eliminated a whole category of special cases where some values had units and some did not.

Second: "here is a dimensionless version of the problem and here is an equivalent with units. The PETSc view of this problem has to be exactly the same." This gave us a clear test criterion. Run the problem both ways. Compare the PETSc vectors. If they match, the non-dimensionalisation is correct. If they do not, something is wrong at a gateway.

Once we had those two principles, the remaining work was straightforward. Not easy, but straightforward.

10. Units in Practice

A complete example putting it together:

```
import underworld3 as uw
import sympy

# Physical parameters
eta_0 = uw.expression("eta_0", uw.quantity(1e21, "Pa*s"))
rho   = uw.expression("rho",   uw.quantity(3300, "kg/m**3"))
g     = uw.expression("g",     uw.quantity(9.8,  "m/s**2"))
alpha = uw.expression("alpha", uw.quantity(3e-5, "1/K"))
DT    = uw.expression("DT",    uw.quantity(1500, "K"))

# Rayleigh number — computed symbolically with units
L = uw.quantity(2900, "km")
kappa = uw.quantity(1e-6, "m**2/s")
Ra = (rho * alpha * g * DT * L**3) / (eta_0 * kappa)
```

```
# Ra is dimensionless — the units cancel exactly
Ra.to_reduced_units()  # ~1e7, dimensionless
```

The Rayleigh number computation is familiar from any geodynamics textbook. The difference is that here, the units are checked automatically. If you accidentally used a velocity where a diffusivity should go, Pint would produce a quantity with leftover dimensions, and downstream code would catch the inconsistency.

This does not replace physical intuition. You still need to know that a Rayleigh number of 10^7 means vigorous convection. But it does replace the mental arithmetic of tracking dimensions through a chain of multiplications and divisions. The computer handles that now.

The Underworld project is supported by AuScope and the Australian Government through the National Collaborative Research Infrastructure Strategy (NCRIS). Source code: github.com/underworldcode/underworld3